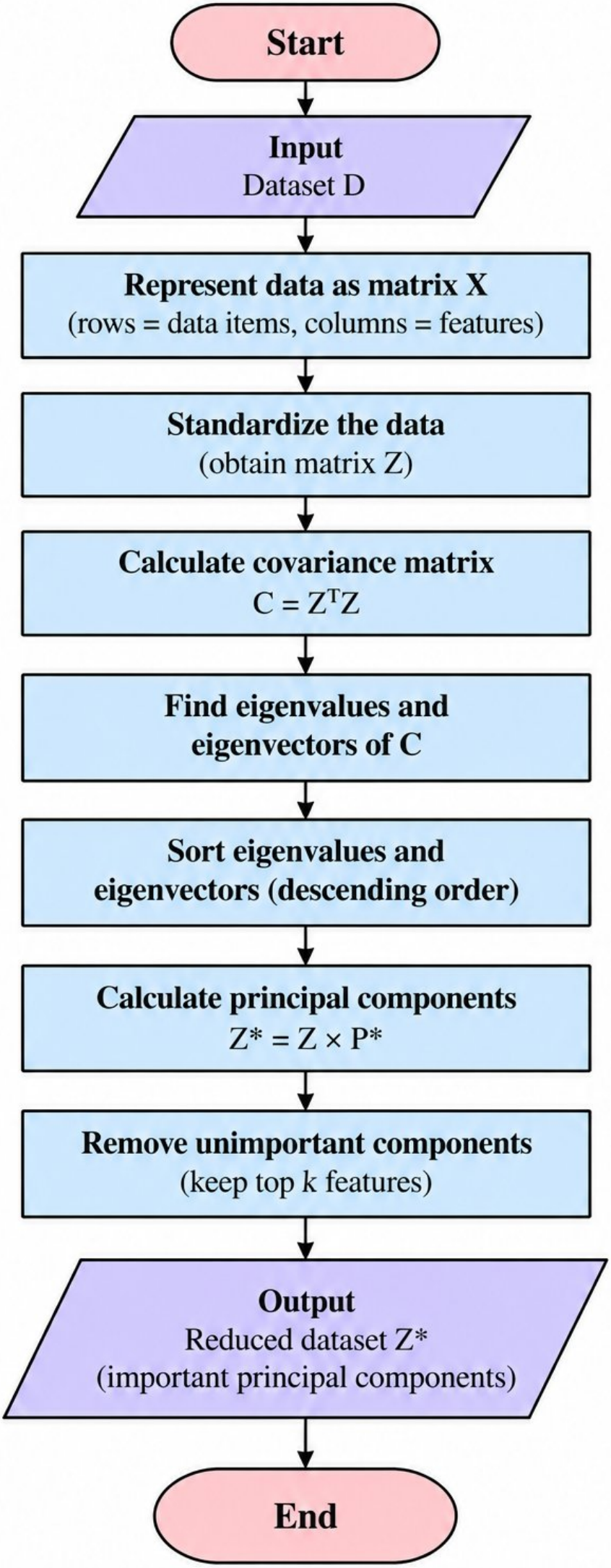
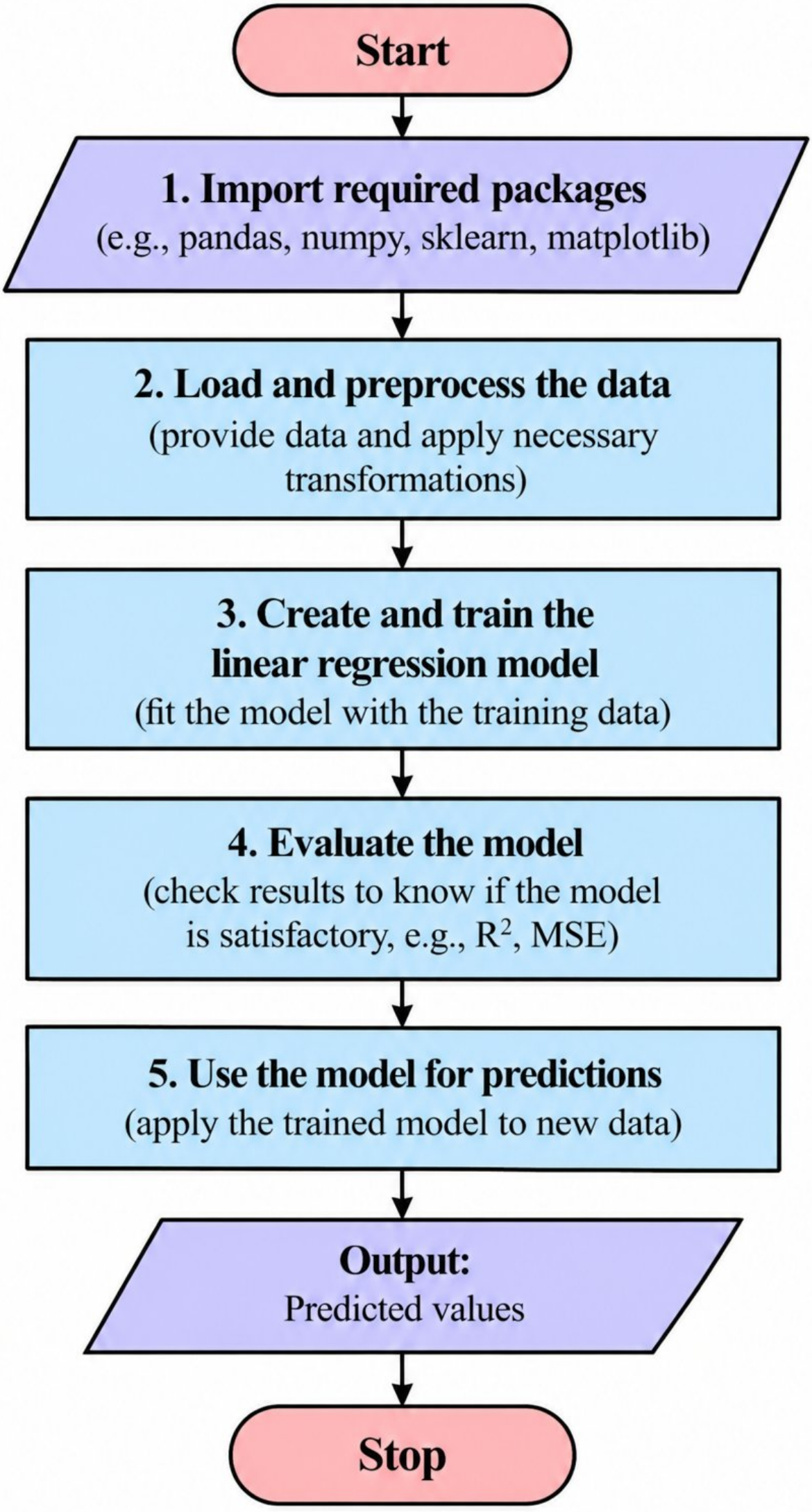
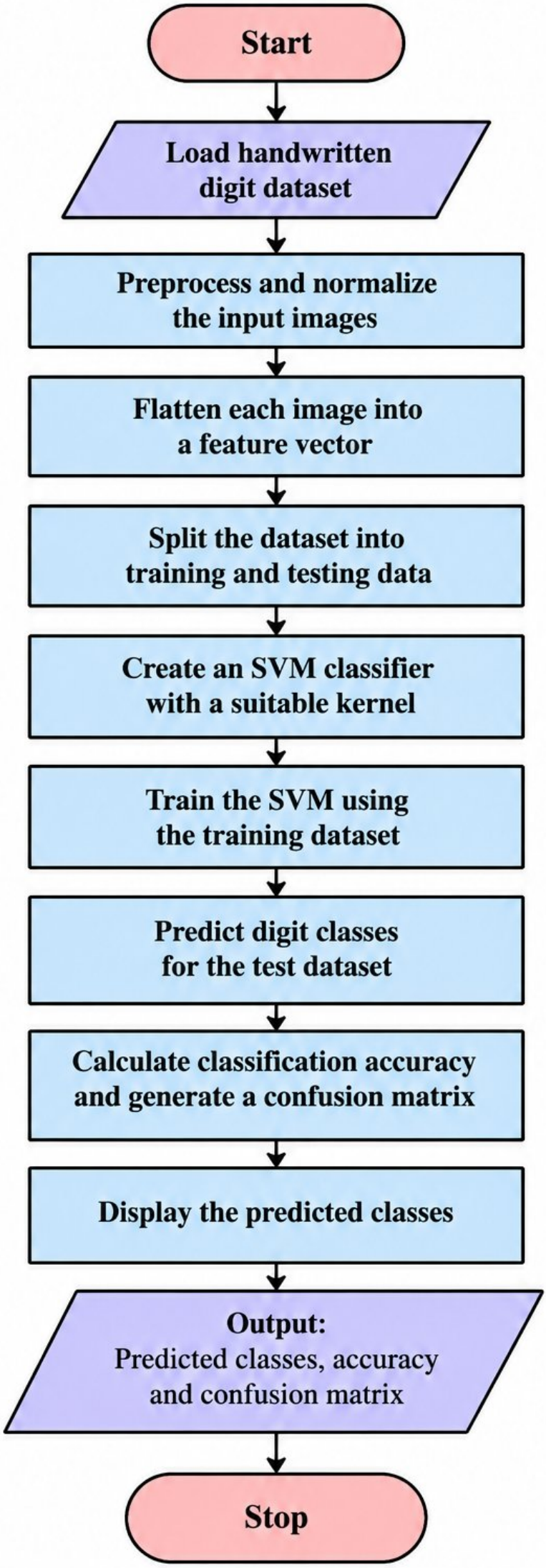
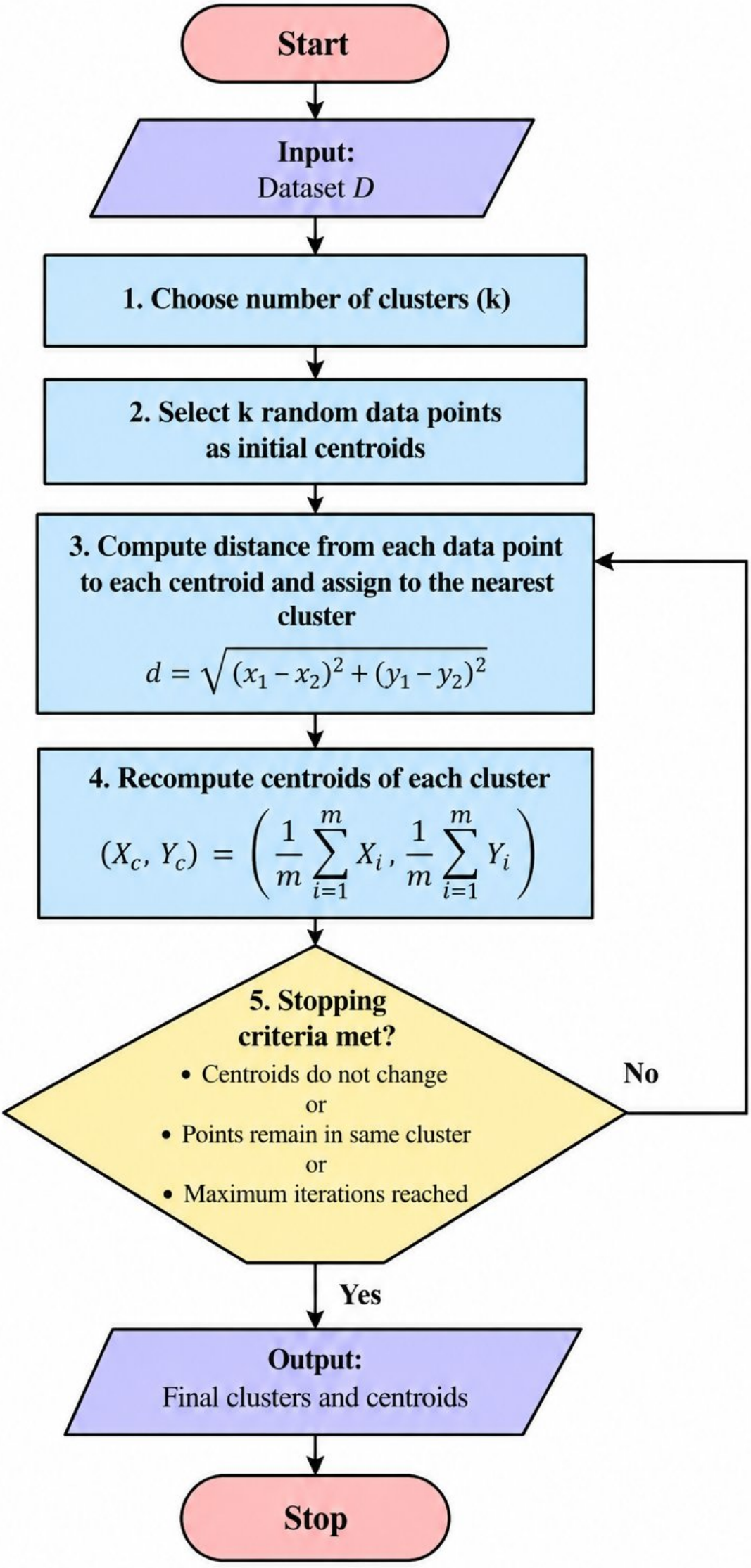
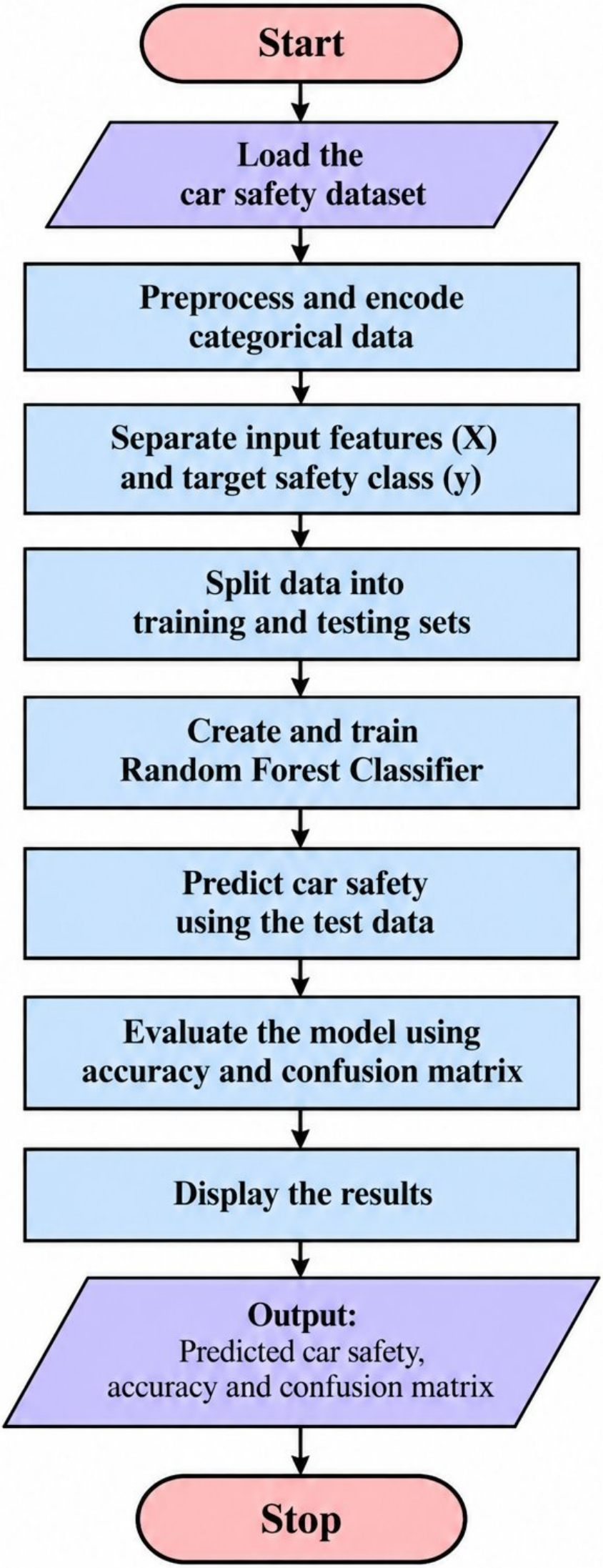
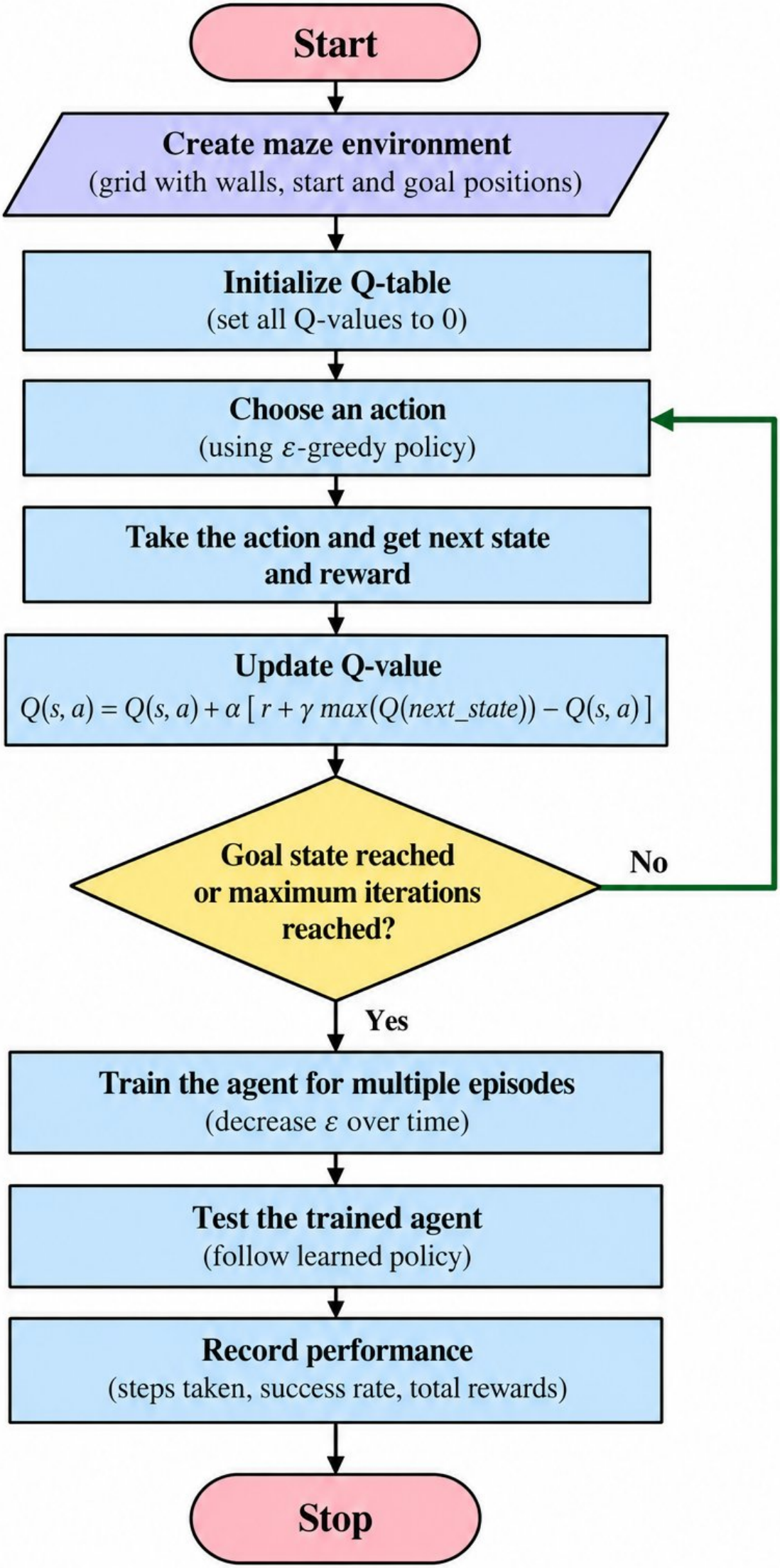

Start
Input
Dataset D
Represent data as matrix X
(rows = data items, columns = features)
Standardize the data
(obtain matrix Z)
Calculate covariance matrix
C = Z^T Z
Find eigenvalues and
eigenvectors of C
Sort eigenvalues and
eigenvectors (descending order)
Calculate principal components
Z* = Z × P*
Remove unimportant components
(keep top k features)
Output
Reduced dataset Z*
(important principal components)
End


Start
1. Import required packages
(e.g., pandas, numpy, sklearn, matplotlib)
2. Load and preprocess the data
(provide data and apply necessary transformations)
3. Create and train the linear regression model
(fit the model with the training data)
4. Evaluate the model
(check results to know if the model is satisfactory, e.g., R^2, MSE)
5. Use the model for predictions
(apply the trained model to new data)
Output:
Predicted values
Stop


Start
Load handwritten digit dataset
Preprocess and normalize the input images
Flatten each image into a feature vector
Split the dataset into training and testing data
Create an SVM classifier with a suitable kernel
Train the SVM using the training dataset
Predict digit classes for the test dataset
Calculate classification accuracy and generate a confusion matrix
Display the predicted classes
Output:
Predicted classes, accuracy and confusion matrix
Stop


Start
Input:
Dataset D
1. Choose number of clusters (k)
2. Select k random data points as initial centroids
3. Compute distance from each data point to each centroid and assign to the nearest cluster
$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$
4. Recompute centroids of each cluster
$(X_c, Y_c) = \left(\frac{1}{m} \sum_{i=1}^{m} X_i , \frac{1}{m} \sum_{i=1}^{m} Y_i \right)$
5. Stopping criteria met?
• Centroids do not change
or
• Points remain in same cluster
or
• Maximum iterations reached
No
Yes
Output:
Final clusters and centroids
Stop


Start
Load the car safety dataset
Preprocess and encode categorical data
Separate input features (X) and target safety class (y)
Split data into training and testing sets
Create and train Random Forest Classifier
Predict car safety using the test data
Evaluate the model using accuracy and confusion matrix
Display the results
Output:
Predicted car safety, accuracy and confusion matrix
Stop


Start
Create maze environment
(grid with walls, start and goal positions)
Initialize Q-table
(set all Q-values to 0)
Choose an action
(using ε-greedy policy)
Take the action and get next state
and reward
Update Q-value
$Q(s, a) = Q(s, a) + \alpha\,[\, r + \gamma\, max(Q(next_state)) - Q(s, a)\,]$
Goal state reached
or maximum iterations
reached?
No
Yes
Train the agent for multiple episodes
(decrease ε over time)
Test the trained agent
(follow learned policy)
Record performance
(steps taken, success rate, total rewards)
Stop